

ECE-412
Spring 2026
Lab 1

Jesus Ruiz Ramos

Report

Demo

Lab 1 Report, Introduction to AVR Instructions for the ATmega328P(B)

Jesus Ruiz Ramos
ECE 412

Abstract

The intent of the lab is to learn about the intricacies of the AVR instruction set and the internal operations of the ATmega328P(B) as well as how it handles memory with its instruction-set. Experiments were conducted on the MPLAB X IDE with the debugging system in order to achieve a deep-dive in how each instruction operated on the microprocessor. The lab itself, has resulted in a better understand of the differences between each of the memory types, how the CPU communicates with them, and how the registers are used to verify data and perform more complex operations through 3 completed laboratory sections.

Body

Introduction

All the following experiments are performed on a Fedora Linux powered Framework 13 laptop using the AMD Ryzen 7840U processor. Because the computer utilizes Linux as its operating system the AVRASM2 could not directly be used as it was never compiled for the Linux kernel and the officially compiled AVR-GCC instruction set used a different syntax. For this reason, AVRA, an open source implementation of the AVRASM2 instruction set is used as it replicates 99% of the AVRASM2 instruction set, Linux-native, with the same syntax, directly implemented into the MPLAB X IDE in place of the AVRASM2 binary and used in junction with the built-in simulation and debugging systems. Experiments are divided into three distinct parts. Part 1 is a review and house tour of the MPLAB X IDE environment and general familiarization with the toolset available as well as installation instructions for preparing the IDE to develop with the AVRASM2 instruction set. Part 2 involves studying and analyzing the directives used to manage memory organization instructions on the microprocessor as well as a deep-dive into the ATmega328P(B)'s memory map. The final discussions involve the AVRASM2's individual instructions and their utilization in the assembly program files.

Assembler Directives

Below are discussed the assembler directives present in **Figure A**, shown below, and how each directive applies to the instruction set as well as their differing traits. The main directives that are discussed include the .cseg, .eseg, .dseg, .org, .exit, .dw, .db, and .byte directives.

Figure A

```
.eseg
.org 0x0
eevar: .dw 0xfaff
msg: .db "HelloWorld"
.dseg
.org 0x100
string: .byte 3
.cseg
.org 0x0
start: ldi r30,56
      ldi r31,24
      add r31,r30
here:  jmp here
      .exit
```

cseg

The cseg instruction, which is both short for and defines the *code segment*, marks the point in the program begins. The code segment is the specific area of memory where the executable instructions of a program are stored and that the instructions following its declaration are to be stored in the program memory for program execution. In Figure A, the cseg instruction is being used to declare the start of the instructions that will be run when the program executes in real time. The instructions belonging to the segment, in this example, are the org declaration, load instructions ldi, the add instruction, and the jump instruction.

eseg

The eseg directive marks the point in the program that includes instructions to be added to the EEPROM memory. The *Electrically Erasable Programmable Read-Only Memory* is defined as a type of non-volatile memory for storing long-persistence data. This segment allows for instructions occurring after it to be stored in this long term memory. Shown in Figure A, the kernel is using the org instruction to set an origin point in memory, and save to non-volatile memory, a hexadecimal value and a string value. These values will persist between uses of the program since the EEPROM memory, by design, can persist even after losing power.

dseg

A data segment contains operations such as memory allocation or reservation for the code segments that follow it. In more abstracted languages, like C, this would be the equivalent to a setup method that initializes values that need to persist throughout the entire run of the code but not beyond the run itself. The AVR instruction set defines dseg to being dedicated to setting byte values using the BYTE directive and uses the ORG instruction to store these bytes in a specific location. In Figure A, the data segment is being used in order to set the the byte value of 3 in address 0x100 prior to the execution of the code segment. If the code segment were to require the byte at this location, it can read it accordingly since it persists for its run but subsequent runs of the program will always reset the value to what the data segment declares.

org

The “set program origin” directive is given within a data segment, EEPROM segment, or a code segment to indicate from which point to start reading memory addresses for each directive’s origins. It does not use its own counter but rather uses the counter of the respective segment that it is initialized in. The counter is what determines which subsequent memory location follows for each set of data that is saved. For segments like EEPROM and code, the default values of the location counters are zero. The user manual draws the distinction that the SRAM location counter defaults to 32. In Figure A, the org directive is used only once per each segment to declare the starting point of memory where each of their respective data is to be stored. eseg was set at 0x0, dseg at 0x100, and cseg at 0x0 overwriting the data set by the eseg with its own.

exit

The exit directive declares the literal exit point of the program. Once the program reaches the exit directive, it stops running. In general, it exits the file immediately rather than awaiting the end of the assembler file which is how the assembler interprets files by default. The exit directive can also be used in conjunction with the include directive which makes it possible to include other assembly files and upon hitting the exit directive, it continues from where it left off on the parent assembly program.

dw

The define constant words directive allocates memory to store literal values in either EEPROM memory or program memory. To effectively use a DW directive, it should be preceded by a label in the format in order for it to be reference elsewhere in the program: `label: .dw "text"` and it can not be declared empty, requiring it to reference at least one expression value. In Figure A, the program is saving the value of 0xfaff which falls within the bounds required by the DW directive as it evaluates to 64255 and is a 16-bit number. Expression lists can be used with the directive and are stored as comma separated values. It's limited to values that evaluate to a number between -32768 and 65535. The DW directive also uses a 16-bit two's complement to store negative values in memory.

db

DB serves a similar function to DW but it is limited to bytes as per its name of define constant bytes. Rather than saving an expression including characters and bytes, it's limited to just bytes and it must be included in a code segment or an EEPROM segment. It too, requires a label in order to be referenced, but unlike the DW directive, it is limited to a value that evaluates to a number between -128 and 255. It uses an 8-bit two's complement to store negative values. In Figure A, the DB directive is being used to declare a literal message with the msg label and the value of "HelloWorld" and saving it to EEPROM memory.

ATmega328P(B) Memory Map

The ATmega328P(B)'s memories consist of the Reprogrammable Flash Program Memory, the SRAM Data Memory, and the EEPROM Data Memory. Each one of these memory types correspond to a memory directive that is part of the AVR instruction set. The .eseg directive is used to assemble data into the EEPROM memory segment, the .dseg directive determines what instructions and data are assembled into the SRAM Data Segment, and the program itself, including the portions that reside within the .cseg directive, resides in the Flash Program memory.

The program storage itself is 32 KB of re-programmable flash memory. It has a boot loader support function used for configuring the load of programs. The flash memory itself is organized as 16K x 16 with a 14 bit-wide program counter addressing it. This is due to the size limit of AVR instructions at 16 or 32 bits width.

The SRAM can be written to in two distinct ways. The primary way to write to the SRAM is using its standard IN and OUT instructions register for the addresses *up to* 0x60. The remaining addresses of the SRAM, 0x60 to 0xFF can only be reached using the ST/STS/STD and LD/LDS/LDD instructions which are slower and take up more registers. The first 32 locations available in the SRAM correspond to the register file, available indirectly with the 0x00 up to 0x1F with the I/O instructions along with the following 64 locations that belong to the standard I/O memory. The remaining locations include the 160 locations that extended I/O registers that can only be accessed via the slower ST/LD instructions and the general SRAM memory space which takes up what remains of the 2K locations in the SRAM. Data access for the SRAM is performed in two clock cycles of the CPU.

The key difference between these is how each reacts to the instructions over a run. EEPROM is the long-term storage location of the 328P(B) chip. It has 1 KB of EEPROM memory which is read/writeable in individual bytes. In order to communicate with the EEPROM memory from the CPU, the AVR instructions need to specify an Address register, data register, and control register, all for the EEPROM memory specifically. Proper use of the EEPROM memory is enforced in order to safely manage it. When reading from the memory, the CPU halts for 4 cycles, and when reading, pauses for 2 before the next instruction is executed.

The general purpose I/O registers are another part of the I/O memory space that allows for storing global variables and status flags within the address range 0x00 and 0x1F and are directly bit-accessible using the SBI, CBI, SBIS, and SBIC instructions.

EEPROM control instructions consist of 3 separate registers. For its address register, with an address offset of 0x21, it uses a 16 bit-wide register. This register is separated into the low-byte EEARL from bits 7 to 0, referred to as the EEARL, and the high-byte EEARH from bits 15 to 8, referred to as the EEARH. When referring to the I/O registers as data space, they need to be offset according to the I/O register address offset of 0x21. Likewise when referring directly to the I/O registers with the IN and OUT commands, the offset is reduced. The actual EEPROM address only takes up bits 9 to 0 specifying the address in the EEPROM's 1KB of space, and a proper value must be written to EEPROM before it can be accessed.

The 0x20 reduced offset when using IN and OUT instructions actually correspond to the EEPROM's data register. This register consists of a byte-wide instruction referred to as EEDR but it is limited due to this size, which is what is preventing it from adequately communicating beyond the first 64 locations. The EEDR contains the data read out from the EEPROM at the address given by the EEAR.

The EEPROM control register, EECR determines *what to do* at the address given by the EEAR. It is a byte-wide address register in which only the bits 5 to 0 are actually used. Bits 5 and 4, the program mode bits, determine what operation should occur. A mode of 00 erase and write, 01 erases only, 10 writes only, and 11 is a reserved command. The remaining 4 bits are additional options for the location. Bit 3, EERIE enables/disables interrupt if the SREG is set, 1 enabling the interrupt and 0 disabling it. Likewise, bit 2, EEMPE, the master write enable, allows the following bit EEPE, write enable, to write data to the addressed location. After the address has been written to, the hardware clears the bit to zero after four clock cycles. This is a requirement in order to allow EEPE to bit to control whether to write to the addressed location. The actual order of the operation is: wait for EEPE to reset

to zero, wait for SPEN to become zero; allowing an interrupt to occur, write a new EEPROM address to the EEAR and write new EEPROM data to the EEDR if needed, write a 1 to EEMPE while writing zero to the EEPE, and then after four cycles on EEMPE, write a 1 to EEPE. This is because the software must check that the flash programming is completed before initiating an EEPROM write, thus the EEPROM can not be programmed during a CPU write to the flash memory. The final bit, EERE, enables read instructions on the address. Mentioned before, this halts the CPU for four cycles before the next instruction is executed.

MPLAB X IDE Debug Discussion

The following discussions involve the code snippet **Figure B** below involving the debugging process and the debugging features that are bundled as part of the MPLAB X IDE toolkit.

Figure B

```
start:  ldi r30,56 ;red highlighted: breakpoint
        ldi r31,24
        add r31,30
here:   jmp here ;green highlighted: pc value
```

Stepping Overview

The shown snippet includes 2 direct load instructions into the I/O registers R30 and R31 with the values of 56 and 24 respectively. Stepping through each line of the assembly program, shows the I/O memory updating these two registers with the values loaded into them once the operation has completed. After `ldi r30,56` the I/O memory window shows a red highlight indicating a change to it as well as the newly added value of 56 on R30 as opposed to the default 0 value. Notably, this line is also labeled “start”, setting it as a jump back point in the program, though it never actually gets used. After the `ldi r31,24` instruction, the following register, R31 is set to the value of 24 and equally highlighted in red instead of the R30 register since it is the latest register to be updated. The `add r31,r30` instruction, performs an addition operation using the values stored in the two registers, and it overrides the value stored in the R31 register. The add instruction specified that the first register/location declared is where the resultant value will be stored. It’s also important to point out that nothing occurs to the R30 register since the add instruction doesn’t affect that register and no operations occur to erase it, so the value remains. Seen in repetitions of the program, the R30 value doesn’t change again and the I/O memory window shows it so by never highlighting the register in red as it’s never changing. Proceeding the addition operation, there is a `jmp` instruction to the “here” label. Because this same line is the line labeled “here”, the program loops in place on this line and nothing else occurs to the program as it spins in an infinite loop until it is forcefully killed by the debug program.

Hex Values

The R30 register contains the hexadecimal value of 0x38 because 38 is the hexadecimal equivalent of the decimal 56. This is $3 * 16 + 8 * 1 = 48 + 8 = 56$. It uses hexadecimal values which are shorter than equivalent binary values.

Program Counter

The program counter increases from 0x000 to 0x003 because the program only runs for 4 lines. It only performs four actions from the starting load operation to the loop. The program counter is doing what it's supposed to which is keep track of the lines of the assembly program that are running and have run.

Manual Program Counter Editing

Setting the program counter value to 0, moves the active program step to the first location in the program memory. This is also shown by the next step that is “stepped into” being the `ldi r31, 24` instruction and updating the PC address value to 0x001 accordingly.

Debug Tools

The first available tools in the debug menu are the “Finish”, “Pause”, “Reset”, and “Continue” options. Finish, requests the end of the debug session and unloads the assembled code from the simulator. This, notably, does not reset the I/O registers on the simulator since these don't actually correspond to the program itself. The Pause function allows you stop the program at its current running step. This allows you to enter a step mode that can be traversed using the Step Into and Step Over functions. The Step Into and Step Over functions are a convention not unique to the MPLAB X IDE and instead are used to differ between entering a portion of an included assembly file and assuming that it has completed and proceeding on the same file, respectively. Stepping Into the next line gives you a more in-depth look to how the assembly is running. The Reset function allows you to restart to the first step of the program section and moves the cursor accordingly. The Continue function unpauses. Outside of the debugger, you can set breakpoints, or lines in the code at which you want the program to pause automatically to review the state of the registers and operations up to that point. These are visually defined with a red square and highlights the corresponding line in red also. Step Out lets you cut the Step Into function short and the Run to Cursor function steps the program automatically until it reaches where you placed your cursor, as a sort of in-debug breakpoint and a short cut for not manually stepping a bunch of times through the program. Set PC to cursor, sets the program counter to be equal to the value of whatever line of the program section the cursor is on and the Focus Cursor at PC function moves the cursor to whatever step the PC is on equally. The debugger also allows you to see how memory changes and operations are occurring through its various windows.

Assmebly Instructions

The following discussion is with respect to all of the AVR instructions that were utilized in this experiment, going over how they work, how they're used, and why they are used.

ldi

Load immediate allows the program to store a data into the data registers of the I/O memory. An `ldi` call requires two parameters that are comma-separated. The first parameter is the register at which the data will be stored, and followed by the value that will be stored. This value, does not necessarily need to be a literal number and can be specified as either a base 10, base 2, or base 16 number in the assembly program. The value inputted can also be an operation between two other values, such as an addition operation, subtraction operation, multiplication, or any of the other logical operators available. `ldi` will overwrite the value stored in the specified address with the one that is passed into as the input parameter.

add

The `add` instruction does exactly what it says. It performs an add operation between two registers but importantly performs so without a carry. This is different from an `adc` operation which occurs with a carry where the added values will be added with no exception for an overflow. It takes less time than the `adc` operator but at the cost of precision. The instruction will also always output the value into what is defined as the `Rd` register, seen as the left parameter register in the assembly program. It does not do anything to the `Rr` register or the right parameter register after the operation concludes and the register remains with the same value as it had prior to the `add` instruction.

sub

The `sub` instruction performs a subtraction operation between two inputted registers in the same format as the `add` instruction and just like the `add` instruction it doesn't raise a carry flag if an overflow occurs and it stores the value in the `Rd` register while keeping the `Rr` register untouched. It will always subtract the value in the `Rr` register from the value in the `Rd` register. It also has related operations like `sbc` which performs a subtraction while raising a carry flag for the overflow.

and

The `and` instruction performs a logical AND operation between two registers. Just like an AND operation, it will go bit by bit and perform the `and` instruction for each one and saving the resultant value in the `Rd` register while leaving the `Rr` register unchanged.

mul

The `mul` instruction will multiply two registers together. Specifically, it performs an unsigned multiplication operation. This means that the last bit 7, operates just as a normal 8 bit number would and it isn't treated as a negative value. The operation requires the input registers of `Rd` and `Rr` just like `add` and `sub` but unlike those operations the resultant value is *always* stored in the `R1` and `R0` registers because the result of multiply two 8-bit numbers results in a 16-bit number. This operation leaves the `Rd` and `Rr` registers unchanged and will overwrite whatever is stored in the `R1` and `R0` registers.

st

The `st` instruction allows you to “save” a value in a register to a SRAM memory indirectly. This means that you don't specify which location but rather the auto-address locations. You use the `X`, `Y`, and `Z` auto-addresses to save the value into a specific memory location according to the address represented by `R27` and `R26` for `X`, for example. The `st` instruction takes first the parameter of the auto-address and then a second parameter which is the address of the value that is going to be stored. It uses the address stored in the `R27` and `R26` registers to create a 16-bit representation for the memory location with the same address. The auto-address parameter can also have a `+` or `-` modifier applied to it. The modifier `+` is added after the auto-address, like `X`, in the form `X+` in order to request the register be stored on the `X` address and then increment the `X` value in `R27:R26` by one. This would set the `X` value to now being the location after which the register was stored. The `-` modifier, written as `-X` first decreases the auto-address value and then stores the value in this location.

clr

The `clr` instruction requests the specified register be cleared. It is, as shown in the AVR user guide, to take in two parameters, though in use, it only takes one. The single parameter it takes is the register that is going to be cleared and have its value reset to default, but it is actually an alias for performing a `eor` operation on the register with the same register which results in a 0.

ser

The `ser` instruction is the exact opposite of the `clr` instruction in that, instead of resetting every bit in the register to 0, it sets every bit to 1. While the `clr` instruction does this by running a `XOR` operation on its own register, the `ser` instruction sets the value to `$FF` which is equivalent to every bit of the register being set to 1, the highest number possible in the 8-bit space.

jmp

The `jmp` instruction directly sets the program counter to a specific value. This value does not need to be a numerical address, it can be specified for jumping to a specific address in the program memory. More often, however, the command is used with a label which represents a specific program counter address. This label is a string that can be reused.

brmi

The brmi instruction is used to jump to a specific program counter address if the operation preceding it results in a negative number. The literal calculation for this is if $(N=1)$ then $PC \leq PC + k + 1$ which means that if the negative flag is raised, then the program counter will be equal to the address after the k addresses after the current value of the program counter. This can be used to set conditions on the program.

nop

The nop instruction stands for “no operation” and instructs the CPU to do just that. This makes the CPU do absolutely nothing for one clock cycle. Theoretically, this can be used to implement systems that require a specific delay. It can also be used as filler while other operations are occurring simultaneously or be used for debugging as well. It takes exactly one clock cycle on the CPU to perform this operation.

breq

The breq or branch if equal operation, is another conditional operation that performs similar to brmi but checking if the previous operation raised the zero flag. The actual calculation is if $(Z=0)$ then $PC \leq PC + K + 1$. It performs the same jump as the brmi but the only differing value is which flag is checked. It has an opposing instruction brne which checks for when the zero flag is low and performs the same jump. They both check for whether the previous operation resulted in a raised zero flag.

lsl

The lsl or logical shift left operation takes a single parameter Rd and shifts every one of its bits down to the left leaving a zero on the least significant bit. This can be used for shortcut mathematical operations on the CPU without directly performing complex operations. The actual operation occurs by utilizing the carry flag as the store for the most significant bit and the setting the bits on the register from the 6:0 bits over to the 7:1 bits and then sets the least significant bit, 0, to the low value since no bit preceded the 0th on the original register.

lsr

The lsr or logical shift right operation does the same thing but instead places the least significant bit in the carry flag, shifts every bit once to the right from registers 7:1 to 6:0, and this time sets the new 7th bit to a low value.

asr

The `asr` is the arithmetic shift right instruction is used to shift signed values one bit to the right. This is why it is the “arithmetic” instruction. It performs a similar operation as `lsl` though in the opposite direction. The least significant bit is saved in the carry flag and then each following bit in the register from 6:1 is shifted one bit to the right. The most significant bit, the 7th bit, isn’t shift right as well because in a signed value, the most significant bit actually represents the negative, so to maintain the same sign, all the values other than the most significant bit are shifted.

bset

The `bset` instruction, meaning bit set flag, is used to set a specific status flag to a high value, or 1. This can be used to manipulate operations and can also be used to forcefully set flags to change how branches operate in the assembly program. It takes an 8-bit number where each individual bit represents one of the possible flags that can be set. The main ones being the carry flag at 0, zero flag at 1, negative flag at 2, and other flags which weren’t discussed yet. The input value must be the 8-bit equivalent of the high values for each of the corresponding flags. To set the zero flag but not the carry flag, for example, the input would need to be 0x02 which is the hex equivalent of 00000010 which sets just the zero flag, or the second bit, to high. The instruction communicates directly and only with the SREG or status register.

bclr

The `bclr` instruction, does the exact opposite by using the same 8-bit bus to set the corresponding bits to low and *reset* the flags to their low values.

Code and Instructions Analysis

Figure C

The value stored in the SRAM location 0x100 is 24. This value comes from the `st X, r30` instruction which places the value stored in R30 to be stored in the SRAM memory location that is represented by the values in the R27:R26 registers. In this case, with the high-byte R27 register equal to the value 1, and R26 the low-byte register equal to zero, they are treated together as a 16-bit number representing the address in SRAM at which the value r30 is to be stored. The r30 value itself comes from the final and operation. First the two values of r31 and r30, 24 and 56 respectively, are added and their result is stored in R31. The actual value of R30 does not change in this operation. The following sub operation reverses this by subtracting R30 from R31 setting R31 back to 24 and again not changing the value in R30 which at this point is still 56. The final logical operation is an and operation from the R31 register into the R30 register. The binary representation of the value in R30, 24, is 011000. The binary representation of the value in R31, 56, is 111000. When these two values are used

to perform a logical and operation, the resulting binary value is 011000 which is equivalent to base 10 24 which is what the R30 register is set to. The `st X, r30` then saves the value in SRAM for future use.

As per the status flags, when debugging the program, the addition instruction results in a half-carry flag, on the 5th bit of the SREG register, because the operation results in the R31 register becoming a value greater than 4 bits, or a nibble. This wasn't triggered by the R30 register already being bigger than 4 bits because the flags only trigger after arithmetic and logical operations. The second flag to be raised is the SREG's second bit which is the zero flag. This was triggered because clearing the value in the R30 register is considered a logical operation and it results in a zero, thus raising the flag. Respectively, the values from the register were set to 32 and 34, respectively, in base 10.

Figure D

Debugging the program allows a more clear view about what is happening after the subtraction. This time, subtracting the r30 from r31 results in a negative value which raises the negative flag, SREG(2), to high. The following line is a branch operation which checks whether the negative flag has been raised. In this case, because the branch has been raised, the operation proceeds to the program counter address outlined by the "here" label where another branch operation is tested for the zero flag this time. However, the instruction that would've set the zero flag to high, `clr r30` was never run because the `brmi here` instruction jumped past it so the `breq` evaluation never returns true thus stepping into the `.exit` directive and ending the program with a crash instead of the usual loop. Had the negative flag *not* been raised, the program would've continued through to the clear instruction, setting the zero flag to high, and making `breq here` run in a loop and never hitting the `.exit` directive.

Figure E

The `lsl r30` instruction is setting the r30 register shifted one bit to the left logically. This operation results in raising the half-carry, sign bit, two's complement, and carry flags. These flags result in the SREG containing a binary value of 0b000111001. The following `lsr r30` instruction performs the same operation in the opposite direction but this time the original most significant bit has been lost after the `lsl` instruction completed. The instruction results in the SREG binary value of 0b00100000 representing **just** the half-carry flag. The following `asr r30` instruction doesn't raise any flags because the value doesn't have a leading 1 most significant bit so it's not treated as a signed value and the second least significant bit is already a zero so the carry flag isn't raised. The following `bset` and `bclr` perform a raise and lower respectively of the SREG(2) bit which correspond to the negative flag. The `brmi` instruction that proceeds it does check for this flag but given that the flag has already been reset to zero, the branch never runs and the program continues as normal.

From this point, 3 distinct `st` instructions are executed on the X auto-address. This auto-address is given by the 16-bit value of R27:R26, but with the X+ modifier, the 16-bit address is incremented each time that the r30 value is saved to the address marked by X. This increment increases the least significant nibble by 1 resulting in R26 increasing from 0x00 to 0x01, then 0x02, and finally 0x03. In

the same vein, each time that the X address is incremented, the value of r30 is then stored in the X address after it. Verifying the SRAM memory state at the locations 0x0100 to 0x0102 shows the value of r30, 16, is stored in all 3 locations.

Source Code (Software)

The following source codes were used for the final portion of the experiments which involved assembling the following programs and debugging them to learn more about their functions at runtime.

Figure C

```
/* Arithmetic, Logic, Data Transfer Instructions */
.cseg
.org 0x0
start: ldi r26,0x00 ;load the hex value 0 in register 26
      ldi r27,0x01 ;load the hex value 1 in register 27
      ldi r30,56   ;load the value 56 in register 30
      ldi r31,24   ;load the value 24 in register 31
      add r31,r30   ;add value in R30 to R31 and store in R31, triggers a half-
carry flag at SREG(5)
      sub r31,r30   ;subtract the value in R30 from R31 and store in R31
      and r30,r31   ;perform a logical AND operation on the values R30 and R31
and store in R31
      st X,r30      ;store the value of R30 in the location R27:R26
      clr r30       ;clear the value in register R30, triggers a zero flag on
SREG(1)
      ser r31       ;set the value of register R31 to all high bits or FF
here:  jmp here     ;jump to the program counter value represented by the here
label
      .exit
```

Figure D

```
/* Conditional Branch Instructions */
.cseg
.org 0x0
start: ldi r26,0x00
      ldi r27,0x01
      ldi r30,56
      ldi r31,24
      sub r31,r30
      brmi here     ;jump to the here pc address if the negative flag is high
      st X,r30      ;store the value of R30 in the location R27:R26
      nop          ;do nothing for one CPU clock cycle
      clr r30       ;clear the r30 register, raise the zero flag
here:  breq here     ;jump to the here pc address if the zero flag is low
      .exit
```

Figure E

```
.cseg
.org 0x0
start: ldi r26,0x00
```



```

        ldi    r27,0x01
        ldi    r30,0xAC
        lsl    r30          ;shift the r30 value to the left logically, SREG set to
0b00111001
        lsr    r30          ;shift the r30 value to the right logically, SREG set to
0b00100000
        asr    r30          ;shift the r30 value to the right arithmateically, flags
don't change
        bset   2            ;set the status register to 0b00000100 setting the negative
flag to 1
        bclr   2            ;set the status register to 0b00000000 setting the negative
flag to 0
        brmi   here
        st     X+,r30       ;store the r30 value in SRAM location 0x0100 and increment
r26 to 0x01
        st     X+,r30       ;store the r30 value in SRAM location 0x0101 and increment
r26 to 0x02
        st     X+,r30       ;store the r30 value in SRAM location 0x0102 and increment
r26 to 0x03
here:    jmp     here
        .exit

```

Schematics

None

Analysis

The results of the discussions posed regarding the AVR instruction set, imply a better understanding of the AVR instruction set as it is implemented in code as well as the physical hardware, providing a deeper understand of how the microprocessor functions with respect to the instructions in its program memory.

Directives Results

The research experiments conducted throughout this lab have outlined the differences between each directive, the different memory types, how the ATmega328P(B) microprocessor's memory is mapped, how the MPLAB X IDE is used to program and debug the AVR instruction set, the instructions that are available, as well as their parameters, while giving practice with analyzing the results of a debug session.

The important resultant directives that have emerged from the experiments involve the declaration of data storage in the specific memory segments including the EEPROM memory segment, the data memory segment, and the code segment of the assembly program. Their respective .eseg, .dseg, and .cseg directives have shown how they allow the assembly to communicate with their respective memories as well as their limitations, including, but not limited to, the .org directive being used by all of the memory segments, the .byte directive being the only way to communicate with the data segment and store data within it, and the EEPROM segment using the .dw and .db directives exclusively to save data to its long-term storage.

Memory Map Results

The discussion regarding the memory map of the ATmega328P(B) has also raised concerns with the limitations of the microprocessor's memory spaces and the limitations surrounding not just their performance but their functionality. The particular detail to highlight is the different sizes of memory spaces available for I/O operations and SRAM for complex operations. The I/O register consists of 32 locations, the extended I/O register has an additional 64 and only these two are accessible via the IN and OUT instructions which are much faster. The following 160 locations and the SRAM's remaining 2K locations can each only be accessed via the slower LD and SD instructions because of how many bytes take up the address space for these memory spaces.

Breakpoints and Debugging Results

Discussions on the MPLAB X IDE's debugging system have also brought up a better understanding of it and how to assess and operate the debugger effectively to get the most out of it. The usage of breakpoints have been crucial in order to automatically pause code snippets that would

otherwise skip to the end and not give a chance to step through the program and verify how it is running.

Part of the key debugging tools also includes a better understanding of how to read and assess the individual I/O registers of the microprocessor as well as how to modify registers such as the program counter register to affect the debugger itself.

Assembly Instructions

Along with a deeper understanding of the directives of the AVR instruction set, the experiments have also aided in the understanding of the AVR instructions themselves. Notable ones are the `ldi` instruction for setting a value in the I/O space, the `st` instruction to store said value in SRAM and the communication between the X auto-address value and the R27 and R26 addresses namely. The discussion of memory access also brought up the Y and Z auto-addresses which were mentioned in the memory map of the ATmega328P(B).

Logical operators such as the add, subtract, and AND operations were discussed as well as how they affect the running program's registers and the relevant flags that they raise such as the zero flag, carry flag, half-carry flag, and negative flags. Also the `bset` and `bclr` instructions to directly manipulate the SREG register which contains the status flags.

Conclusion

The purpose of the experiment was to gain a better understanding of the AVR instruction set and become familiarized with the MPLAB X IDE which is used to program the ATmega328P(B). Ultimately, this was achieved, as the experiment has turned up ample information regarding not just the instructions themselves but their limitations, their connections to the physical microprocessor and how they can be observed within the IDE's debug system. Further research can be conducted regarding the instructions as a large portion of them is not reviewed by this report. However, future research experiments will be made easier with the compounded knowledge that is available in this report from assessing this small portion.

References

Microchip Technology Inc. 2018. AVR® Microcontroller with Core Independent Peripherals and PicoPower® Technology, ATmega328P(B)

Microchip Technology Inc. 2022. MPLAB® X IDE User's Guide